Routing 라우팅 시스템

개요

IPFS의 라우팅 시스템은 분산 네트워크에서 콘텐츠와 피어를 찾는 핵심 메커니즘입니다. 라우팅은 크게 두 가지 유형으로 나뉩니다: **콘텐츠 라우팅**(Content Routing)과 **피어 라우팅**(Peer Routing). 콘텐츠 라우팅은 특정 CID를 제공하는 피어를 찾는 것이고, 피어 라우팅은 특정 피어 ID의 주소 정보를 찾는 것입니다.

Boxo는 다양한 라우팅 구현체를 제공하며, DHT(Distributed Hash Table), HTTP 위임 라우팅, 오프라인 라우팅 등을 지원합니다.

콘텐츠 라우팅과 피어 라우팅

콘텐츠 라우팅 (Content Routing)

콘텐츠 라우팅은 특정 CID를 제공할 수 있는 피어들을 찾는 과정입니다.

```
package main


import (
"context"
"fmt"
"log"
"github.com/ipfs/go-cid"
"github.com/libp2p/go-libp2p/core/routing"
```

```
)


// 콘텐츠 라우팅 기본 인터페이스
type ContentRouting interface {
// 특정 CID를 제공하는 피어들을 찾습니다
FindProvidersAsync(ctx context.Context, cid cid.Cid, count int) <-chan peer.Add
// 자신이 특정 CID를 제공한다고 네트워크에 알립니다
Provide(ctx context.Context, cid cid.Cid, announce bool) error
}


func contentRoutingExample(router routing.ContentRouting) {
ctx := context.Background()
// 찾고자 하는 CID
targetCID, err := cid.Decode("bafkqac3jobxhgidsn5rww4yk")
if err != nil {
log.Fatal(err)
}
// 해당 CID를 제공하는 피어들을 비동기적으로 찾기
providers := router.FindProvidersAsync(ctx, targetCID, 10) // 최대 10개
fmt.Printf("CID %s를 제공하는 피어들:\n", targetCID)
for provider := range providers {
fmt.Printf(" 피어: %s\n", provider.ID)
fmt.Printf(" 주소: %v\n", provider.Addrs)
```

```
}

// 자신이 해당 CID를 제공한다고 알리기

err = router.Provide(ctx, targetCID, true)

if  err != nil {

log.Printf("제공자 등록 실패: %v", err)

} else {

fmt.Printf("CID %s 제공자로 등록 완료\n", targetCID)

}

}
```

피어 라우팅 (Peer Routing)

피어 라우팅은 특정 피어 ID의 주소 정보를 찾는 과정입니다.

```
import (

"github.com/libp2p/go-libp2p/core/peer"

)


// 피어 라우팅 기본 인터페이스

type PeerRouting interface {

// 특정 피어의 주소 정보를 찾습니다

FindPeer(ctx context.Context, id peer.ID) (peer.AddrInfo, error)

}
```

```
func peerRoutingExample(router routing.PeerRouting) {

ctx := context.Background()

// 찾고자 하는 피어 ID

targetPeerID, err := peer.Decode("12D3KooWGC6TvWhfapngX6wvJHMYVKpDMXGb9ZJ2E5EHcU

if err != nil {

log.Fatal(err)

}

// 피어 정보 찾기

peerInfo, err := router.FindPeer(ctx, targetPeerID)

if err != nil {

log.Printf("피어 찾기 실패: %v", err)

return

}

fmt.Printf("피어 %s 정보:\n", targetPeerID)

fmt.Printf(" 주소들: %v\n", peerInfo.Addrs)

}
```

DHT와 위임된 라우팅

DHT (Distributed Hash Table) 라우팅

DHT는 분산 해시 테이블을 사용하여 콘텐츠와 피어 정보를 저장하고 검색하는 방식입니다.

```
import (
```

```go
dht "github.com/libp2p/go-libp2p-kad-dht"
"github.com/libp2p/go-libp2p/core/host"
)

func createDHTRouter(h host.Host) (*dht.IpfsDHT, error) {
// DHT 라우터 생성
kdht, err := dht.New(context.Background(), h)
if err != nil {
return nil, err
}
// DHT 부트스트랩 (네트워크 연결)
err = kdht.Bootstrap(context.Background())
if err != nil {
return nil, err
}
return kdht, nil
}

func dhtExample(h host.Host) {
ctx := context.Background()
// DHT 라우터 생성
kdht, err := createDHTRouter(h)
if err != nil {
```

```
log.Fatal(err)

}

defer kdht.Close()

// 콘텐츠 제공자로 등록

testCID, _ := cid.Decode("bafkqac3jobxhgidsn5rww4yk")

err = kdht.Provide(ctx, testCID, true)

if err != nil {

log.Printf("DHT 제공자 등록 실패: %v", err)

}

// 제공자 찾기

providers := kdht.FindProvidersAsync(ctx, testCID, 5)

for provider := range providers {

fmt.Printf("DHT에서 찾은 제공자: %s\n", provider.ID)

}

// 피어 찾기

targetPeer, _ := peer.Decode("12D3KooWGC6TvWhfapngX6wvJHMYVKpDMXGb9ZJ2E5EHcUGBfM

peerInfo, err := kdht.FindPeer(ctx, targetPeer)

if err != nil {

log.Printf("DHT 피어 찾기 실패: %v", err)

} else {

fmt.Printf("DHT에서 찾은 피어 정보: %v\n", peerInfo.Addrs)

}
```

```
}
```

HTTP 위임 라우팅 (Delegated Routing)

HTTP 위임 라우팅은 HTTP API를 통해 라우팅 서비스를 제공하는 방식입니다.

```
import (
"github.com/ipfs/boxo/routing/http/client"
"github.com/ipfs/boxo/routing/http/server"
"github.com/ipfs/boxo/routing/http/types"
"github.com/ipfs/boxo/routing/http/types/iter"
)

// HTTP 라우팅 클라이언트 생성
func createHTTPRoutingClient(serverURL string) (*client.Client, error) {
// 기본 클라이언트 생성
c, err := client.New(serverURL)
if err != nil {
return nil, err
}
return c, nil
}

// 고급 옵션을 가진 HTTP 라우팅 클라이언트
```

```
func  createAdvancedHTTPClient(serverURL  string, identity crypto.PrivKey, peerI

options := []client.Option{

// 신원 설정 (제공자 등록용)

client.WithIdentity(identity),

// 제공자 정보 설정

client.WithProviderInfo(peerID, addrs),

// 프로토콜 필터 설정

client.WithProtocolFilter([]string{"transport-bitswap"}),

// 주소 필터 설정

client.WithAddrFilter([]string{"ip4", "ip6"}),

// 스트리밍 결과 필수 설정

client.WithStreamResultsRequired(),

// 사용자 정의 HTTP 클라이언트

client.WithHTTPClient(&http.Client{

Timeout: 30 * time.Second,

}),

// 사용자 에이전트 설정

client.WithUserAgent("MyApp/1.0"),

}

return  client.New(serverURL, options...)

}



func  httpRoutingClientExample() {
```

```
ctx := context.Background()

// HTTP 라우팅 클라이언트 생성

c, err := createHTTPRoutingClient("https://cid.contact")

if err != nil {

log.Fatal(err)

}

// 제공자 찾기

targetCID, _ := cid.Decode("bafkqac3jobxhgidsn5rww4yk")

providersIter, err := c.FindProviders(ctx, targetCID)

if err != nil {

log.Fatal(err)

}

defer providersIter.Close()

fmt.Printf("HTTP 라우팅에서 찾은 제공자들:\n")

for providersIter.Next() {

result := providersIter.Val()

if result.Err != nil {

log.Printf("오류: %v", result.Err)

continue

}

// 레코드 타입에 따른 처리

switch record := result.Val.(type) {

case *types.PeerRecord:
```

```
fmt.Printf(" 피어: %s, 프로토콜: %v\n", record.ID, record.Protocols)

default:

fmt.Printf(" 알 수 없는 레코드 타입: %T\n", record)

}

}

// 피어 찾기

targetPeer, _ := peer.Decode("12D3KooWGC6TvWhfapngX6wvJHMYVKpDMXGb9ZJ2E5EHcUGBfM

peersIter, err := c.FindPeers(ctx, targetPeer)

if err != nil {

log.Fatal(err)

}

defer peersIter.Close()

fmt.Printf("HTTP 라우팅에서 찾은 피어 정보:\n")

for peersIter.Next() {

result := peersIter.Val()

if result.Err != nil {

log.Printf("오류: %v", result.Err)

continue

}

peerRecord := result.Val

fmt.Printf(" 피어: %s\n", peerRecord.ID)

fmt.Printf(" 주소: %v\n", peerRecord.Addrs)

fmt.Printf(" 프로토콜: %v\n", peerRecord.Protocols)
```

```
}

}
```

HTTP 라우팅 서버 구현

```
import (

"net/http"

"github.com/ipfs/boxo/routing/http/server"

)


// 사용자 정의 콘텐츠 라우터 구현

type MyContentRouter struct {

// 실제 구현에서는 데이터베이스나 다른 저장소 사용

providers  map[string][]types.Record

peers  map[string]*types.PeerRecord

}


func  NewMyContentRouter() *MyContentRouter {

return &MyContentRouter{

providers: make(map[string][]types.Record),

peers: make(map[string]*types.PeerRecord),

}

}
```

```go
func (cr *MyContentRouter) FindProviders(ctx context.Context, c cid.Cid, limit

records, exists := cr.providers[c.String()]

if !exists {

return  iter.FromSlice([]iter.Result[types.Record]{}), nil

}

// 제한 적용

if  limit > 0 && len(records) > limit {

records = records[:limit]

}

// 결과를 iter.Result로 변환

results := make([]iter.Result[types.Record], len(records))

for  i, record := range  records {

results[i] = iter.Result[types.Record]{Val: record}

}

return  iter.FromSlice(results), nil

}


func (cr *MyContentRouter) FindPeers(ctx context.Context, pid peer.ID, limit  in

peerRecord, exists := cr.peers[pid.String()]

if !exists {

return  iter.FromSlice([]iter.Result[*types.PeerRecord]{}), nil

}
```

```
results := []iter.Result[*types.PeerRecord]{
{Val: peerRecord},
}
return iter.FromSlice(results), nil
}


func (cr *MyContentRouter) ProvideBitswap(ctx context.Context, req *server.Bitsw
// 제공자 정보를 저장소에 저장
for _, key := range req.Keys {
peerRecord := &types.PeerRecord{
Schema: types.SchemaPeer,
ID: &req.ID,
Addrs: make([]types.Multiaddr, len(req.Addrs)),
Protocols: []string{"transport-bitswap"},
}
for i, addr := range req.Addrs {
peerRecord.Addrs[i] = types.Multiaddr{Multiaddr: addr}
}
cr.providers[key.String()] = append(cr.providers[key.String()], peerRecord)
cr.peers[req.ID.String()] = peerRecord
}
// 권장 TTL 반환
return req.AdvisoryTTL, nil
```

```
}

func (cr *MyContentRouter) GetIPNS(ctx context.Context, name ipns.Name) (*ipns.R
// IPNS 레코드 조회 구현
return  nil, routing.ErrNotFound
}

func (cr *MyContentRouter) PutIPNS(ctx context.Context, name ipns.Name, record *
// IPNS 레코드 저장 구현
return  nil
}

func  createHTTPRoutingServer() http.Handler {
// 콘텐츠 라우터 생성
contentRouter := NewMyContentRouter()
// 서버 옵션 설정
options := []server.Option{
// 레코드 제한 설정
server.WithRecordsLimit(100),
// 스트리밍 레코드 제한 설정
server.WithStreamingRecordsLimit(1000),
// 라우팅 타임아웃 설정
server.WithRoutingTimeout(30 * time.Second),
```

```
// Prometheus 메트릭 레지스트리 설정
server.WithPrometheusRegistry(prometheus.DefaultRegisterer),
}
// HTTP 핸들러 생성
handler := server.Handler(contentRouter, options...)
return  handler
}


func  httpRoutingServerExample() {
// HTTP 라우팅 서버 생성
handler := createHTTPRoutingServer()
// HTTP 서버 시작
fmt.Println("HTTP 라우팅 서버 시작: http://localhost:8080")
log.Fatal(http.ListenAndServe(":8080", handler))
}
```

라우팅 설정과 최적화

오프라인 라우팅

```
import (
"github.com/ipfs/boxo/routing/offline"
"github.com/ipfs/go-datastore"
```

```go
"github.com/libp2p/go-libp2p-record"
)

func createOfflineRouter() routing.Routing {
// 메모리 데이터스토어 생성
ds := datastore.NewMapDatastore()
// 검증자 생성 (IPNS 레코드 검증용)
validator := record.NamespacedValidator{
"ipns": ipns.Validator{},
}
// 오프라인 라우터 생성
offlineRouter := offline.NewOfflineRouter(ds, validator)
return offlineRouter
}

func offlineRoutingExample() {
ctx := context.Background()
router := createOfflineRouter()
// 오프라인 모드에서는 네트워크 작업이 실패합니다
testCID, _ := cid.Decode("bafkqac3jobxhgidsn5rww4yk")
// 제공자 등록 시도 (실패)
err := router.Provide(ctx, testCID, true)
if err != nil {
```

```
fmt.Printf("예상된 오프라인 오류: %v\n", err)
}
// 제공자 찾기 시도 (빈 결과)
providers := router.FindProvidersAsync(ctx, testCID, 10)
count := 0
for range providers {
count++
}
fmt.Printf("오프라인 모드에서 찾은 제공자 수: %d\n", count)
// 로컬 레코드 저장/조회는 가능
key := "/test/key"
value := []byte("test value")
err = router.PutValue(ctx, key, value)
if err != nil {
log.Printf("값 저장 실패: %v", err)
} else {
fmt.Println("오프라인 모드에서 값 저장 성공")
}
retrievedValue, err := router.GetValue(ctx, key)
if err != nil {
log.Printf("값 조회 실패: %v", err)
} else {
fmt.Printf("오프라인 모드에서 조회한 값: %s\n", string(retrievedValue))
```

```
}
}
```

제공자 쿼리 매니저

```
import (
"github.com/ipfs/boxo/routing/providerquerymanager"
)

func  createProviderQueryManager(dialer providerquerymanager.ProviderQueryDialer
options := []providerquerymanager.Option{
// 최대 동시 요청 수 설정
providerquerymanager.WithMaxInProcessRequests(5),
// 최대 제공자 수 설정
providerquerymanager.WithMaxProviders(20),
// 타임아웃 설정
providerquerymanager.WithMaxTimeout(30 * time.Second),
// 무시할 피어들 설정
providerquerymanager.WithIgnoreProviders(/* 무시할 피어 ID들 */),
}
return  providerquerymanager.New(dialer, router, options...)
}
```

```go
func providerQueryManagerExample(h host.Host, router routing.ContentDiscovery)
ctx := context.Background()
// 제공자 쿼리 매니저 생성
pqm, err := createProviderQueryManager(h, router)
if err != nil {
log.Fatal(err)
}
defer pqm.Close()
// 제공자 비동기 검색
testCID, _ := cid.Decode("bafkqac3jobxhgidsn5rww4yk")
providers := pqm.FindProvidersAsync(ctx, testCID, 10)
fmt.Printf("제공자 쿼리 매니저로 찾은 제공자들:\n")
for provider := range providers {
fmt.Printf(" 피어: %s\n", provider.ID)
fmt.Printf(" 주소: %v\n", provider.Addrs)
}
}
```

복합 라우팅 시스템

```go
// 여러 라우팅 시스템을 조합한 복합 라우터
type CompositeRouter struct {
```

```
primary routing.Routing // 주 라우터 (예: DHT)

secondary routing.Routing // 보조 라우터 (예: HTTP)

offline routing.Routing // 오프라인 라우터

}


func  NewCompositeRouter(primary, secondary, offline routing.Routing) *Composite

return &CompositeRouter{

primary: primary,

secondary: secondary,

offline: offline,

}

}


func (cr *CompositeRouter) FindProvidersAsync(ctx context.Context, c cid.Cid, co

resultChan := make(chan peer.AddrInfo, count)

go  func() {

defer  close(resultChan)

// 여러 라우터에서 동시에 검색

primaryChan := cr.primary.FindProvidersAsync(ctx, c, count)

secondaryChan := cr.secondary.FindProvidersAsync(ctx, c, count)

seen := make(map[peer.ID]bool)

found := 0

// 두 채널에서 결과를 병합
```

```
for found < count {
select {
case provider, ok := <-primaryChan:
if !ok {
primaryChan = nil
} else if !seen[provider.ID] {
seen[provider.ID] = true
resultChan <- provider
found++
}
case provider, ok := <-secondaryChan:
if !ok {
secondaryChan = nil
} else if !seen[provider.ID] {
seen[provider.ID] = true
resultChan <- provider
found++
}
case <-ctx.Done():
return
}
// 두 채널이 모두 닫혔으면 종료
if primaryChan == nil && secondaryChan == nil {
```

```
break

}

}

}()

return resultChan

}


func (cr *CompositeRouter) Provide(ctx context.Context, c cid.Cid, announce boo

// 모든 라우터에 제공자 등록

var errors []error

if err := cr.primary.Provide(ctx, c, announce); err != nil {

errors = append(errors, fmt.Errorf("primary router: %w", err))

}

if err := cr.secondary.Provide(ctx, c, announce); err != nil {

errors = append(errors, fmt.Errorf("secondary router: %w", err))

}

// 오프라인 라우터는 제공자 등록을 지원하지 않으므로 생략

if len(errors) > 0 {

return fmt.Errorf("provide errors: %v", errors)

}

return nil

}
```

```
func (cr *CompositeRouter) FindPeer(ctx context.Context, id peer.ID) (peer.AddrI

// 순차적으로 시도

if info, err := cr.primary.FindPeer(ctx, id); err == nil {

return info, nil

}

if info, err := cr.secondary.FindPeer(ctx, id); err == nil {

return info, nil

}

return cr.offline.FindPeer(ctx, id)

}



// 기타 라우팅 인터페이스 메서드들 구현...

func (cr *CompositeRouter) PutValue(ctx context.Context, key string, value []by

// 모든 라우터에 값 저장

var errors []error

if err := cr.primary.PutValue(ctx, key, value, opts...); err != nil {

errors = append(errors, err)

}

if err := cr.secondary.PutValue(ctx, key, value, opts...); err != nil {

errors = append(errors, err)

}

if err := cr.offline.PutValue(ctx, key, value, opts...); err != nil {

errors = append(errors, err)
```

```
}
if len(errors) == 3 {
return fmt.Errorf("all routers failed: %v", errors)
}
return nil
}


func (cr *CompositeRouter) GetValue(ctx context.Context, key string, opts ...ro
// 순차적으로 시도
if value, err := cr.primary.GetValue(ctx, key, opts...); err == nil {
return value, nil
}
if value, err := cr.secondary.GetValue(ctx, key, opts...); err == nil {
return value, nil
}
return cr.offline.GetValue(ctx, key, opts...)
}


func (cr *CompositeRouter) SearchValue(ctx context.Context, key string, opts ..
// 첫 번째로 성공하는 라우터의 결과 반환
if ch, err := cr.primary.SearchValue(ctx, key, opts...); err == nil {
return ch, nil
}
```

```
if ch, err := cr.secondary.SearchValue(ctx, key, opts...); err == nil {
return ch, nil
}
return cr.offline.SearchValue(ctx, key, opts...)
}


func (cr *CompositeRouter) Bootstrap(ctx context.Context) error {
// 모든 라우터 부트스트랩
var errors []error
if err := cr.primary.Bootstrap(ctx); err != nil {
errors = append(errors, err)
}
if err := cr.secondary.Bootstrap(ctx); err != nil {
errors = append(errors, err)
}
if err := cr.offline.Bootstrap(ctx); err != nil {
errors = append(errors, err)
}
if len(errors) > 0 {
return fmt.Errorf("bootstrap errors: %v", errors)
}
return nil
}
```

```
func  compositeRoutingExample(dhtRouter, httpRouter, offlineRouter routing.Routi

ctx := context.Background()

// 복합 라우터 생성

compositeRouter := NewCompositeRouter(dhtRouter, httpRouter, offlineRouter)

// 제공자 등록

testCID, _ := cid.Decode("bafkqac3jobxhgidsn5rww4yk")

err := compositeRouter.Provide(ctx, testCID, true)

if  err != nil {

log.Printf("복합 라우터 제공자 등록 실패: %v", err)

}

// 제공자 찾기

providers := compositeRouter.FindProvidersAsync(ctx, testCID, 10)

fmt.Println("복합 라우터로 찾은 제공자들:")

for  provider := range  providers {

fmt.Printf(" 피어: %s\n", provider.ID)

}

}
```

실제 사용 예제

완전한 라우팅 시스템 구축

```go
package main


import (
"context"
"fmt"
"log"
"time"
"github.com/ipfs/boxo/routing/http/client"
"github.com/ipfs/boxo/routing/offline"
"github.com/ipfs/go-cid"
"github.com/ipfs/go-datastore"
dsync  "github.com/ipfs/go-datastore/sync"
"github.com/libp2p/go-libp2p"
dht  "github.com/libp2p/go-libp2p-kad-dht"
"github.com/libp2p/go-libp2p-record"
"github.com/libp2p/go-libp2p/core/host"
"github.com/libp2p/go-libp2p/core/peer"
"github.com/libp2p/go-libp2p/core/routing"
)


// RoutingSystem은 완전한 라우팅 시스템을 관리합니다
type RoutingSystem struct {
```

```
host host.Host

dht *dht.IpfsDHT

httpClient *client.Client

offlineRouter routing.Routing

composite *CompositeRouter

}



func  NewRoutingSystem(ctx context.Context) (*RoutingSystem, error) {

// libp2p 호스트 생성

h, err := libp2p.New()

if  err != nil {

return  nil, err

}

// DHT 라우터 생성

kdht, err := dht.New(ctx, h)

if  err != nil {

return  nil, err

}

// HTTP 라우팅 클라이언트 생성

httpClient, err := client.New("https://cid.contact")

if  err != nil {

return  nil, err

}
```

```
// 오프라인 라우터 생성
ds := dsync.MutexWrap(datastore.NewMapDatastore())
validator := record.NamespacedValidator{
"ipns": ipns.Validator{},
}
offlineRouter := offline.NewOfflineRouter(ds, validator)
// 복합 라우터 생성
composite := NewCompositeRouter(kdht, httpClient, offlineRouter)
rs := &RoutingSystem{
host: h,
dht: kdht,
httpClient: httpClient,
offlineRouter: offlineRouter,
composite: composite,
}
// DHT 부트스트랩
go func() {
err := kdht.Bootstrap(ctx)
if err != nil {
log.Printf("DHT 부트스트랩 실패: %v", err)
}
}()
return rs, nil
```

```
}

func (rs *RoutingSystem) Close() error {
rs.dht.Close()
return rs.host.Close()
}

func (rs *RoutingSystem) ProvideContent(ctx context.Context, c cid.Cid) error {
return rs.composite.Provide(ctx, c, true)
}

func (rs *RoutingSystem) FindProviders(ctx context.Context, c cid.Cid, maxProvid
providers := rs.composite.FindProvidersAsync(ctx, c, maxProviders)
var result []peer.AddrInfo
for provider := range providers {
result = append(result, provider)
}
return result
}

func (rs *RoutingSystem) FindPeer(ctx context.Context, id peer.ID) (peer.AddrInf
return rs.composite.FindPeer(ctx, id)
}
```

```
func (rs *RoutingSystem) GetStats() map[string]interface{} {
stats := make(map[string]interface{})
// DHT 통계
if rs.dht != nil {
stats["dht_peers"] = len(rs.dht.RoutingTable().ListPeers())
}
// 호스트 통계
stats["connected_peers"] = len(rs.host.Network().Peers())
stats["host_id"] = rs.host.ID().String()
stats["addresses"] = rs.host.Addrs()
return stats
}


func main() {
ctx := context.Background()
// 라우팅 시스템 생성
rs, err := NewRoutingSystem(ctx)
if err != nil {
log.Fatal(err)
}
defer rs.Close()
fmt.Printf("라우팅 시스템 시작됨. 호스트 ID: %s\n", rs.host.ID())
```

```
// 잠시 대기 (DHT 부트스트랩 시간)

time.Sleep(5 * time.Second)

// 테스트 CID

testCID, _ := cid.Decode("bafkqac3jobxhgidsn5rww4yk")

// 콘텐츠 제공자로 등록

fmt.Println("콘텐츠 제공자로 등록 중...")

err = rs.ProvideContent(ctx, testCID)

if  err != nil {

log.Printf("제공자 등록 실패: %v", err)

} else {

fmt.Println("제공자 등록 완료")

}

// 제공자 찾기

fmt.Println("제공자 검색 중...")

providers := rs.FindProviders(ctx, testCID, 5)

fmt.Printf("찾은 제공자 수: %d\n", len(providers))

for  i, provider := range  providers {

fmt.Printf(" 제공자 %d: %s\n", i+1, provider.ID)

}

// 시스템 통계 출력

stats := rs.GetStats()

fmt.Println("\n시스템 통계:")

for  key, value := range  stats {
```

```
    fmt.Printf(" %s: %v\n", key, value)

    }

    // 계속 실행

    fmt.Println("\n라우팅 시스템이 실행 중입니다. Ctrl+C로 종료하세요.")

    select {}

    }
```

모범 사례

1. 라우팅 전략 선택

- **DHT**: 완전 분산, 높은 가용성, 느린 속도
- **HTTP 위임**: 빠른 속도, 중앙화된 서비스 의존
- **복합**: 여러 전략 조합으로 장점 극대화

2. 성능 최적화

- 적절한 타임아웃 설정
- 동시 요청 수 제한
- 캐싱 전략 활용
- 네트워크 조건에 따른 동적 조정

3. 안정성 확보

- 여러 라우팅 소스 활용
- 장애 복구 메커니즘

- 모니터링 및 알림 시스템
- 점진적 백오프 전략

4. 보안 고려사항

- 신뢰할 수 있는 라우팅 서버 사용
- 레코드 검증 강화
- 스팸 및 악의적 피어 필터링
- 프라이버시 보호

문제 해결

일반적인 오류와 해결책

1. **제공자를 찾을 수 없음**

- 네트워크 연결 상태 확인
- DHT 부트스트랩 상태 점검
- 라우팅 서버 가용성 확인

2. **느린 라우팅 응답**

- 타임아웃 값 조정
- 여러 라우팅 소스 병렬 사용
- 로컬 캐싱 구현

3. **DHT 연결 문제**

- 부트스트랩 노드 상태 확인
- 방화벽 및 NAT 설정 점검
- 피어 연결 수 모니터링

4. **HTTP 라우팅 오류**

- 서버 URL 정확성 확인
- 인증 정보 검증
- 네트워크 프록시 설정 확인

5. **메모리 사용량 증가**

- 캐시 크기 제한 설정
- 정기적인 정리 작업 수행
- 메모리 프로파일링 실행